

Video RAG Retrieval Project

<https://github.com/aiyshwary/Video-RAG-Retrieval>

OVERVIEW

Python **FAISS** **OpenCLIP** **ViT-GPT2** **Sentence Transformers** **RAG**
A semantic video search system that extracts frames, generates scene captions using ViT-GPT2, and en

IMPACT

Built a Visual RAG system that makes unstructured video content semantically searchable: a text query r

TECHNICAL WALKTHROUGH

Python FAISS OpenCLIP ViT-GPT2 Sentence Transformers RAG
indexes via cosine similarity.

“I built a Visual Retrieval-Augmented Generation (RAG) system that retrieves relevant video scenes based on a user’s text prompt by combining vision models and vector similarity search.”

Problem statement

“Searching videos using text is hard because video content is unstructured. Traditional keyword search doesn’t understand what’s actually happening inside a video. My project solves this by converting both video scenes and user queries into embeddings and retrieving the most relevant video segments.”

High-level architecture

“The system has three main components:

Scene understanding using vision models

Vector storage for text and image embeddings

Similarity-based retrieval using VDMS”

Video fi Scene Description + Frames fi Embeddings fi Vector DB

fi Retrieval fi Video Clips

Input & preprocessing

“The input to the system is one or more videos. Each video is processed to extract:

Scene-level textual descriptions using open-source vision models like Video-LLaMA

Key video frames along with metadata like frame number, FPS, and timestamp”*

Vision understanding

“I use vision-language models to understand the video content. These models analyze video scenes and generate natural language descriptions that

summarize what's happening in each scene.”

“For example, a scene might be described as ‘a person walking into a store and picking up an item’.”

Embedding strategy (very important for interviewers)

Aiyshwarya Aruchamy | Portfolio Project github.com/aiyshwaryaVideo RAG Retrieval Project — continue

“To support efficient retrieval, I store two types of embeddings in separate vector stores:”

Text embeddings

Model: SentenceTransformer – all-MiniLM-L6-V2 / L12-V2

Used for:

Scene descriptions

User queries

“This converts textual descriptions into dense vectors that capture semantic meaning.”

Image embeddings

Model: OpenCLIP – ViT-g-14

Used for:

Video frames

“This helps capture visual semantics like objects, actions, and context directly from frames.”

Vector storage & retrieval

“Both text and image embeddings are stored in VDMS, which allows fast vector similarity search. I maintain separate vector collections for text and image embeddings but link them using video metadata.”

Stored metadata includes:

Video name

Scene ID

Frame number

Timestamp

Query flow

“When a user enters a text prompt:”

Prompt is converted into a text embedding

Vector similarity search is performed against:

Scene text embeddings

(Optionally) image embeddings

The system retrieves:

Most relevant scenes

Corresponding frames and timestamps

Final output:

Video name

Scene description

Exact time range to jump to in the video
(question)

“Text embeddings capture semantic meaning, while image embeddings capture visual features. Using both improves retrieval accuracy because some queries are better answered by visual similarity and others by textual context.”

Example:

“‘a red car’ fi image embeddings

‘a tense conversation’ fi text embeddings”

Technical depth

Efficient vector search using cosine similarity

Aiyshwarya Aruchamy | Portfolio Project github.com/aiyshwaryaVideo RAG Retrieval Project — continue

Decoupling embeddings for scalability

Model choice based on speed vs accuracy

Frame sampling using FPS to avoid redundancy

Closing line (impact)

“This approach makes video content searchable, explainable, and scalable, and it can be extended to surveillance, media indexing, or enterprise video search.”

Short version

“I built a Visual RAG system that enables semantic search over videos. It uses vision models to generate scene descriptions and extract frames, converts them into text and image embeddings using SentenceTransformers and OpenCLIP, stores them in VDMS, and retrieves relevant video segments based on user queries.”

Extracts frames at a configurable FPS and generates natural-language scene captions using

-

ViT-GPT2 image captioning.

Computes text embeddings with SentenceTransformers (MiniLM) and image embeddings with

-

OpenCLIP for dual-modality indexing.

Stores embeddings and metadata in two parallel FAISS indexes supporting independent text or

-

image-based retrieval.

Query interface accepts a natural language string and returns top-k matching scenes with frame

-

references and timestamps.

Modular architecture (preprocessing, embeddings, storage, retrieval) designed for extension to

- production vector databases.

Aiyshwarya Aruchamy | Portfolio Project github.com/aiyshwarya

KEY DETAILS

1. Extracts frames at a configurable FPS and generates natural-language scene captions using ViT-GPT2.
2. Computes text embeddings with SentenceTransformers (MiniLM) and image embeddings with OpenCLIP.
3. Stores embeddings and metadata in two parallel FAISS indexes supporting independent text or image search.
4. Query interface accepts a natural language string and returns top-k matching scenes with frame references.
5. Modular architecture (preprocessing, embeddings, storage, retrieval) designed for extension to production vector databases.
6. FAISS indexes use IndexFlatIP with L2-normalized vectors for cosine-similarity search; OpenCLIP uses ViT-GPT2 for image embeddings.
7. Includes a pytest suite and a self-contained demo that synthesizes a 30-frame test video with OpenCV.